Triage Guard

ארכיטקטורת AI נוירו-סימבולית לטריאז' בחדר מיון

עידן בק · ברק אמיר

הבעיה

בחדר מיון עמוס, חולה במצב קשה יכול לקבל בטעות סיווג של "לא דחוף". הוא מחכה בתור, מצבו מחמיר, ואף אחד לא בודק אותו שוב בזמן.

המשתמשים

אחות טריאז' ממלאת טופס ומציעה רמת דחיפות.

אחות אחראית ואחראי משמרת מכריעים ומאשרים שחרור.

טכנאי מטפל בתקלות.

המטופלים הם הנושא של המערכת, לא המפעילים שלה.

הסיכונים שהמערכת מצמצמת

מטופל קריטי שממתין כאילו הוא שגרתי, פרטים מזהים של מטופל שדולפים, והחלטה שאין לה תיעוד או הסבר.

הערך המערכתי

היום

אחות אחת, תחת עומס, קובעת את רמת הדחיפות של כל מטופל. טעות מתגלה מאוחר, וההערכה החוזרת תלויה בזיכרון של הצוות.

עם Triage Guard

- ה-AI נותן הצעה שנייה, לבדיקה צולבת מול האחות.
- אם הפער ביניהם גדול, עובר להחלטה אנושית.
- כל מטופל שממתין מקבל טיימר להערכה חוזרת. ככל שהוא דחוף יותר, הטיימר קצר יותר.

מדדי הצלחה

0 מטופלים שמחכים אחרי מטופל פחות דחוף מהם

- אף מטופל לא נשכח
- אף מטופל לא מתחיל טיפול בלי בדיקת מדיניות ואישור נדרש.
- אף פרט מזהה לא מגיע ל-AI (מייל, שם, ת"ז...)
- כל פעולה נרשמת ביומן המקרה, עם הסבר.

למה לא רק כללים?

ההחלטה כמה מטופל דחוף דורשת שיקול דעת קליני, ואין לה רשימה סגורה של חוקים.

למה לא רק AI?

ה-AI יכול לטעות, הוא לא נותן דין וחשבון, ואפשר לנסות "לשכנע" אותו.

ה-AI רק מציע.
הכללים מחליטים.
בן אדם מאשר כשזה חשוב.

הטכנולוגיות, ולמה בחרנו בכל אחת

טכנולוגיה	מה היא עושה בשבילנו	מתי היא רצה
LangGraph	כל מעבר בין שלבים מוגדר מראש, אז המערכת לא יכולה לאלתר. המצב נשמר במסד הנתונים אחרי כל שלב, ואם המערכת נעצרת, המקרה ממשיך בדיוק מאותה נקודה.	זמן ריצה, בכל שלב
מודל AI (LLM)	קורא את הנתונים הרפואיים בלבד ומציע רמת דחיפות. זה השלב היחיד שדורש שיקול דעת.	זמן ריצה, פעם אחת לכל סיווג
OPA	מנוע המדיניות. קובע מה מותר לעשות ומה מותר להעביר ל-AI. המדיניות כתובה בקובץ נפרד מהקוד.	זמן ריצה, לפני כל פעולה רגישה
Prolog	בודק שהרשומה של המקרה לא גוררת איתה מצבים או עובדות סותרים וקובע מי מורשה להכריע בשער. מחזיר את שם הכלל שנשבר, וממנו נבנה ההסבר שהאחות רואה.	זמן ריצה
Datalog	כל 5 שניות בודק את כל המקרים והטיימרים יחד: מטופל שממתין בלי טיימר, או טיימר פעיל של מקרה שכבר נסגר.	זמן ריצה
Z3	מוכיח מתמטית שהכללים נכונים לכל קלט אפשרי, במקום לבדוק רק כמה דוגמאות.	זמן תכנון, לפני שיש מטופל אחד
Postgres	מסד הנתונים. שומר כל מקרה, כל טיימר ואת יומן המקרה, כך ששום דבר לא הולך לאיבוד.	זמן ריצה, תמיד
Langfuse	מציג כל ריצה שלב אחרי שלב, כך שרואים איפה משהו האט או נכשל.	זמן ריצה, ברקע
FastAPI	שרת קטן ומהיר לכל שירות (ה-board ומערכת הרשומות).	זמן ריצה

מחזור החיים של מקרה

המסלול הרגיל של מטופל

1 **קליטה** — האחות ממלאת טופס ומציעה רמת דחיפות

2 **הכנה** — שליפת היסטוריה והסרת פרטים מזהים

3 **סיווג** — ה-AI מציע רמה, מ-1 (הכי דחוף) עד 5

4 **בדיקת בטיחות** — הכללים בודקים שהרשומה הגיונית

5 **תור** — לפי דחיפות, ואז לפי שעת הגעה

6 **הערכה חוזרת** — כשהטיימר מגיע, אחות מזינה מדדים חדשים והמקרה חוזר לסיווג

7 **סגירה** — המטופל משוחרר והמקרה נסגר

מצב התחלתי: `intake_received` · מצב סופי: `case_closed`

ארבעה מצבים מחוץ למסלול הרגיל

ממתין לאישור אנושי

מתי: פער גדול בין האחות ל-AI, בדיקת בטיחות שנכשלה, או AI לא בטוח בתשובה.

מה קורה: המקרה נעצר עד שאחות אחראית מחליטה.

חסום (BLK)

מתי: מישהו ניסה פעולה שלא מותרת.

מה קורה: הפעולה נדחית, המקרה לא זז, והסירוב נרשם עם הסבר.

כשל

מתי: רק כשהשלב שמסיר פרטים מזהים קורס (באג בקוד). רכיב שנופל (AI, OPA, Prolog, CRM) לא מגיע לכאן, לכל אחד יש חלופה.

מה קורה: המקרה נעצר והטכנאי מקבל התראה. אחרי שהטכנאי מסמן "תוקן", השלב רץ שוב, ואם הוא עדיין שבור המקרה נעצר שוב. אפשר לשחרר את המטופל בכל רגע.

נדחה

מתי: טופס שלא ניתן לרשום (בלי מזהה מקרה תקין), או מקרה כפול לאותו מטופל.

מה קורה: המקרה מסתיים מיד ולא נכנס למערכת.

טבלת מעברים (חלקית)

מצב נוכחי	Event	תנאי (Guard)	מי בודק	מצב הבא
parsing	DATA_PARSED	כל השדות קיימים והקלט תקין	Schema	data_parsed
parsing	INVALID_INPUT_DETECTED	הטופס לא תואם את המבנה הצפוי	Schema	input_rejected
classifying	ACUITY_PROPOSED	הפלט של ה-AI עבר בדיקה	בדיקת פלט	acuity_proposed
acuity_proposed	אוטומטי	פער של 0 או 1 מהאחות	Python + Z3 (בזמן תכנון)	safety_validating
acuity_proposed	אוטומטי	פער של 2 ומעלה	Python + Z3 (בזמן תכנון)	awaiting_human_approval
safety_validating	VERDICT_PROPOSED	בדיקת הבטיחות עברה	Prolog	verdict_proposed
safety_validating	VERDICT_PROPOSED	בדיקת הבטיחות נכשלה	Prolog	awaiting_human_approval
awaiting_human_approval	APPROVAL_RESPONSE_RECEIVED	מי שעונה הוא אחות אחראית	Prolog	safety_validating
monitoring / reassessment_required	MOVE_REQUESTED	עבר בדיקה, אושר, תפקיד מורשה; דילוג דורש סיבה	OPA	treatment_started
כל מצב פעיל	RELEASE_REQUESTED	סיבת שחרור תקינה ותפקיד של אחראית	OPA	case_closed
כל מצב	ACTION_DENIED	תנאי כלשהו נכשל	OPA, Prolog, תנאי בגרף	**BLK, נשאר במקום**

הגרף הוא הטבלה

כל מעבר מותר מוגדר כנתון ב-LangGraph. מעבר שלא בטבלה פשוט לא קיים.

אין קיצור דרך לטיפול

מקרה שלא עבר בדיקת בטיחות ואישור לא יכול להגיע לטיפול.

תנאי שנכשל מחזיר חסימה

המקרה נשאר במקום, והסירוב נרשם ביומן עם השכבה שחסמה.

כללים לאורך זמן

חלק מהכללים לא עוסקים ברגע אחד, אלא בסדר של האירועים. במפרט יש 22 כללים כאלה. הנה הדוגמאות המרכזיות:

תמיד (Safety)

אף מטופל לא נמצא בתור אחרי מטופל פחות דחוף ממנו.

```
G(acuity(a) < acuity(b) → key(a) < key(b))
```

כניסה לטיפול רק אחרי בדיקת בטיחות ואישור אנושי.

```
G(enter_treatment → safety_passed ∧ approved)
```

טיפול מתחיל פעם אחת בלבד.

```
G(treatment_started → X G ¬treatment_started)
```

בסוף, בתוך זמן קצוב (Liveness)

מטופל בתור עובר הערכה חוזרת בזמן, לפי רמת הדחיפות.

```
G(enter_queue → F≤T reassessment_required)
```

מקרה שאף אחד לא טיפל בו מוסלם, ושוב מוסלם, עד שמישהו פועל.

```
G(waiting ∧ ¬acted → F≤T escalated)
```

אחרי כמה סבבי תיקון שנכשלו, המקרה עובר לאחראית משמרת.

```
G(correction_rounds ≥ N → F handed_up)
```

רצפים אסורים

כניסה לתור אחרי בדיקת בטיחות שנכשלה, בלי תיקון ובדיקה חוזרת.

```
safety_failed → enter_queue
```

שדה לא מאושר או טקסט חופשי שמגיע ל-AI.

```
unapproved_field → model_call
```

שינוי במקרה אחרי שהוא נסגר.

```
case_closed → field_changed
```

איך אוכפים: לפני כל צעד, בדיקות ומדיניות חוסמות מעבר אסור. טיימרים מוודאים ש"בסוף" באמת קורה. אחרי הריצה, בדיקה נפרדת (trace check) קוראת את כל יומן המקרה ומסמנת את הרשומה המדויקת שבה הופר אחד משש כללי הרצף.

מקרא: `G` = תמיד · `F` = בסוף · `X` = בשלב הבא · `F≤T` = בסוף, בתוך זמן T

ה-AI מציע, הכללים מחליטים

החלק הנוירוני (AI)

- מודל AI שמציע רמת דחיפות.
- קורא תלונה, מדדים חיוניים והיסטוריה, בלי פרטים מזהים.
- מחזיר רמה (מ-1 עד 5) וציון ביטחון.

הצעה ←

החלק הסימבולי (כללים)

- ארבע מנועים: OPA, Z3, Prolog, Datalog.
- בודקים הרשאות, פרטיות, עקביות וסדר תור.
- מחליטים אם מותר להמשיך.

מה ה-AI אף פעם לא מחליט

רמת הדחיפות הסופית, הסדר בתור, הרשאות, טיפול בפרטים מזהים, תחילת טיפול ושחרור.

כשה-AI והאחות לא מסכימים

- אותה רמה ← נשארים איתה.
- הפרש של רמה אחת ← הרמה של האחות.
- הפרש של 2 ומעלה ← אחות אחראית מחליטה, וההחלטה שלה סופית.
- ה-AI לא בטוח (ביטחון מתחת ל-70%) ← אחות אחראית מחליטה.

כשה-AI מחזיר תשובה לא תקינה

תשובה שלא בפורמט או מחוץ לטווח 5 עד 1 נזרקת, וה-AI נשאל שוב. אחרי שני כישלונות ממשיכים עם הרמה של האחות, והמקרה מסומן. רמה שסותרת את הרשומה נתפסת בבדיקת הבטיחות (Prolog) ועוברת לאחות אחראית.

מנועי הכללים

כלל ברזל (fail-closed): מנוע שלא מצליח לענות לא מאשר. חוסר תשובה נחשב "לא".

OPA

זמן ריצה

האם מותר לבצע את הפעולה עכשיו?

```
allow if {
  input.action == "move"
  safety_settled
  input.case.approved == true
  input.actor_role in move_roles
  order_settled
}
```

השפעה על המעבר: אם התשובה "לא", הפעולה נחסמת (BLK), והמקרה לא עובר ל-`treatment_started`.

Z3

זמן תכנון

האם קיים קלט כלשהו ששובר את הכללים?

```
solver = Solver()
solver.add(Not(claim))
if solver.check() == unsat:
    return True, ""    # PROVED
```

unsat = אין קלט ששובר את הכלל, והכלל מוכח. **sat** = Z3 מחזיר את הדוגמה, והבדיקות נכשלות. כך מובטח שכל פער מוביל למעבר אחד בדיוק.

Prolog

זמן ריצה

האם הרשומה של המקרה הגיונית?

```
violation(auto_resolved_on_major_gap) :-
    acuity_source(auto_resolved),
    gap(G), integer(G), G >= 2.
```

השפעה על המעבר: הפרה מעבירה את המקרה ל-`awaiting_human_approval`, עם משפט שמסביר מה לתקן.

Datalog

זמן ריצה

האם יש מטופל שממתין בלי טיימר לבדיקה חוזרת?

```
unwatched(C) <=
    waiting(C) & ~live_timer(C, 'reassessment', T)
```

השפעה על המעבר: אף מקרה לא זז. ממצא נשלח לטכנאי, והטיימרים לא משתנים.

המוניטור: איך אף מטופל לא נשכח

אף אחד לא יושב מול שעון לכל מטופל. לכן המערכת קובעת לכל מקרה "שעונים מעוררים" (טיימרים), ותהליך רקע מפעיל אותם בזמן.

המערכת לא מחזיקה תהליך פתוח לכל מטופל שממתין. היא שומרת את המצב של המקרה ב-Postgres ועוצרת אותו. כשהטיימר מגיע, או כשאחות לוחצת על משהו ב-board, המקרה ממשיך בדיוק מהנקודה שבה עצר.

כל 5 שניות, המוניטור:

1. מדווח "אני חי", כדי שאפשר יהיה לזהות שהוא קרס.
2. מוצא טיימרים שהגיע זמנם, וגם טיימרים שנכשלו או נתקעו, ומנסה אותם שוב.
3. מעביר כל טיימר להחלטה: Prolog מוודא שהוא מסכים (אם לא, הטיימר נעצר), ו-OPA מאשר רגע לפני הפעולה.
4. מריץ בדיקת Datalog על כל המקרים והטיימרים: מטופל שממתין בלי טיימר, או טיימר פעיל של מקרה שנסגר. מה שנמצא עובר לטכנאי.

טיימר	מתי	מה הוא עושה
הערכה חוזרת	כל 10 דקות ברמה 1, ועד 120 דקות ברמה 5	מעיר את המקרה לבדיקה מחדש
תזכורת לשער אישור	אחרי 10 דקות, ושוב אחרי 20	מזכיר לאחות, ואז לכל האחיות האחראיות
תזכורת לבכיר	10 דקות אחרי הסלמה	מזכיר לכל אחראי המשמרת

הגנות: טיימר שלא ברור אם פעל נבדק קודם ביומן, ואחרי 3 ניסיונות אדם מקבל התראה. טיימר שנכשל 3 פעמים ברצף מוסלם לטכנאי. לכל היותר 5 התראות בשעה לכל קבוצת נמענים.

איך בדקנו את המערכת

מה בודקים	מה זה אומר
מצבים תקינים	כל מעבר בטבלה קיים בגרף, והמקרה מגיע למצב הצפוי.
מצבים אסורים	פעולה לא מורשית נחסמת, והמקרה לא זז.
מדיניות (OPA)	התחלת טיפול, שחרור, מה מותר להעביר ל-AI, ומה קורה כש-OPA נופל.
הוכחות (Z3)	רצועות הפער וסדר התור. בנוסף, מזינים ל-Z3 קוד שגוי בכוונה ובודקים שהוא תופס אותו.
הסקות (Prolog ו-Datalog)	תפקידים, סתירות ברשומה, ומטופל שממתין בלי טיימר.
כללים לאורך זמן	בדיקת העקבות על יומן המקרה, טיימרים והסלמות.
ה-AI טועה	ה-AI מציע רמה שגויה או לא בטוח בתשובה, והכללים עוצרים אותו.
הסברים	לכל פעולה שנחסמה יש סיבה כתובה, עם השכבה שחסמה.
תקלות	כל רכיב שנופל, והמערכת ממשיכה בדרך הבטוחה.

1,000+
בדיקות אוטומטיות עוברות

22 / 22
כללי Temporal Logic נבדקים בקוד

6 / 6
הוכחות Z3 מוכחות

סיכונים, אבטחה וממשל

סיכון	מה מגן מפניו
ה-AI מציע רמה שגויה	ה-AI רק מציע. התשובה נבדקת לפורמט, לטווח ולסתירות, ופער גדול עובר לאחות אחראית.
דליפת פרטים מזהים	רק רשימה מאושרת של שדות מגיעה ל-AI, ו-OPA בודק אותה. אם OPA דוחה אותה, היא לא נשלחת ל-AI: ממשיכים עם הרמה של האחות, והטכנאי מקבל התראה. פרט מזהה שהוקלד מוחלף בסימון [REDACTED] לפני ה-AI ובכל מה שנשלח ל-Langfuse. המקרה עצמו שומר רק מזהה פנימי, והפרטים המזהים נשמרים רק במערכת הרשומות של בית החולים (CRM).
פעולה בלי הרשאה	כל פעולה נבדקת מול התפקיד שמוצהר בבקשה (עדיין אין רשימת צוות בשרת). ברירת המחדל של OPA היא "אסור". רק אחות אחראית או אחראית משמרת יכולות לשחרר.
אין תיעוד	כל שינוי וכל סירוב נרשמים ביומן המקרה, במבנה קבוע. בדיקת היסטוריה מוודאת שהיומן רק גדל: שינוי או מחיקה של רשומה מתגלים ומוצגים.
סודות ומפתחות	מפתחות API והגדרות נשמרים בקובץ ENV (משתני סביבה), ולא בקוד.
החלטה בלי בן אדם	Human-in-the-loop: בנקודות רגישות המקרה נעצר, ונשמר, עד שאחות אחראית מחליטה. אחרי כמה סבבי תיקון שנכשלו, המקרה עובר לאחראית משמרת.

מה קורה כשרכיב נופל

העיקרון: רכיב שנופל אף פעם לא גורם למערכת לאשר בלי בדיקה, ואף פעם לא מונע ממטופל להגיע לטיפול. כשרכיב שמאשר פעולות לא זמין, **אחראית משמרת חותמת במקומו**, והקוד עדיין בודק את העובדות.

הרכיב שנפל	מה המערכת עושה
מודל ה-AI	ממשיכים עם הרמה שהאחות קבעה. המקרה מסומן לבדיקה חוזרת מאוחר יותר.
OPA	הנתונים לא נשלחים ל-AI, כי אי אפשר להוכיח שהם נקיים. להתחלת טיפול ולשחרור נדרשת חתימה של אחראית משמרת.
Prolog	לבדיקת הבטיחות אין תשובה, אז המקרה עובר לשער. אפשר לבדוק שוב כשהרכיב חוזר, או שאחראית משמרת מאשרת להמשיך בלי הבדיקה. זה נרשם כ"אושר על ידי אחראית משמרת", אף פעם לא כ"עבר בדיקה".
המוניטור	מקרים ממשיכים להגיע לתור, אבל הטיימרים ממתינים, והצוות עוקב ידנית. כשהמוניטור חוזר, הוא מפעיל את כל הטיימרים שהתעכבו.

כשרכיב נופל, רואים את זה ב-board בשלושה מקומות: בכרטיס של כל מקרה שעבר בלי הרכיב יש סימון, בפאנל של המקרה יש תיבה שמפרטת מה חסר, ובחלון הפעולה כתוב מי צריך לחתום במקום (למשל: "OPA לא זמין, העברה ושחרור דורשים אחראית משמרת").

מה עובד, מה עדיין דמו, ומה הלאה

עובד היום

- מכונת המצבים ב-LangGraph, השער האנושי, החסימות ויומן המקרה
- 4 מנועי הכללים, עם fail-closed
- המוניטור והטיימרים, שנשמרים גם אם המערכת נופלת
- ה-board לאחיות, והמעקב ב-Langfuse

עדיין דמו

- מערכת הרשומות (CRM) מכילה 20 מטופלים מדומים
- הקליטה היא טופס מובנה, בלי סריקת מסמכים או טקסט חופשי
- ההרשאות הן לפי תפקיד בלבד

הצעדים הבאים

- לבדוק את ה-AI על מקרים אמיתיים (אנונימיים) ולמדוד דיוק
- ספים כמו ביטחון 70%, מספר סבבי תיקון וזמני הערכה חוזרת דורשים נתונים ואישור קליני
- להוסיף ניהול גרסאות למדיניות למנועי הכללים
- מערכת משתמשים (אחיות)

איך המערכת מונעת מה-AI להוביל מקרה לרצף מסוכן או למצב אסור?

ה-AI יכול רק להציע, וכל מעבר מוגבל לחצים שהוגדרו מראש. מעל זה יש ארבע שכבות:

1 מראש

מנוע Z3 מוכיח שהכללים עצמם לא שבורים.

2 לפני כל צעד

המנועים OPA ו-Prolog חוסמים פעולה אסורה.

3 בזמן ריצה

המוניטור דואג לבדיקה חוזרת בזמן, ו-Datalog בודק את כל המערכת.

4 אחרי הריצה

בדיקת העקבות קוראת את היומן ומסמנת כל הפרה של ששת כללי הרצף.

כל סירוב מחזיר את המקרה לאישור אנושי או עוצר אותו, ואף פעם לא מקדם אותו.